

Exercise: Capstone Build — mCase Configuration Dictionary

Rules of Engagement

1. **Agent mode only** — your planning is done; now you build
 2. **One checkpoint at a time** — review before moving forward
 3. **Start a fresh conversation at each checkpoint** — paste your project rules as context
 4. **Reference your rules** — start each chat with `@project-rules.mdc`
 5. **Review before continuing** — if it doesn't work or look right, fix it before moving on
-

Setup: MCP Credentials

Add a `.env` file to your project root:

```
MCASE_API_URL=<provided by instructor>
MCASE_API_USER=<provided by instructor>
MCASE_API_PASSWORD=<provided by instructor>
```

Verify the MCP is working before starting CP1. Open a new Cursor chat and ask:

```
Use the mCase MCP to list all available datalists.
```

You should see a list of datalist names. If not, check your MCP configuration in Cursor settings.

Checkpoint 1 — Datalist Browser (20 min)

What you're building:

- A page that lists all datalists: Label, SystemName, FieldCount
- Clicking a datalist fetches its full definition and shows its fields
- Fields show: Label, addType (field type), required flag

Starting prompt (paste into a new Agent mode chat):

```
@project-rules.mdc

Building the mCase configuration dictionary.

Checkpoint 1: Datalist browser
- Use the mCase MCP `schema` tool to fetch all datalists
- Display as a list: Label, SystemName, FieldCount
```

- Clicking a datalist uses `datalist\_by\_name` to fetch its full definition
- Display fields for the selected datalist: Label, addType (type), required flag

Step 1: Show me the data shape returned by each MCP call before writing any UI.

Step 2: Build the data fetching layer. Pause for my review.

Step 3: After I confirm the data layer, build the UI.

Review checklist before moving to CP2:

- ☐ App runs and data loads
- ☐ Datalist names are human-readable (Label, not SystemName)
- ☐ Field types are visible (addType values)
- ☐ You can tell which datalist you've selected

Checkpoint 2 — Field Detail (20 min)

What you're building:

- Field type badges (based on addType)
- Picklist values shown inline for picklist fields
- Required fields visually distinguished
- Field descriptions shown (if present)

Starting prompt (new conversation):

@project-rules.mdc

Checkpoint 2: Field detail enhancement

The FieldDefinition shape is:

```
{ fieldId, label, systemName, addType, required, hidden, isMirror, values[],
source, description }
```

Enhance the field display:

- Add a type badge to each field (addType: "Text", "Date", "Picklist", "Dynamic", etc.)
- For fields with values[], show the picklist options inline or in a collapsed section
- Visually distinguish required fields (bold, asterisk, or a badge)
- Show the description field if present

Do not change the data fetching layer from CP1. Only modify the field rendering. Pause after updating field rendering for my review.

Review checklist before moving to CP3:

- ☐ You can visually scan field types across a datalist
- ☐ Picklist values are accessible without drilling further

- ☐ Required fields are obvious
 - ☐ Performance is acceptable (no obvious slowdowns)
-

Checkpoint 3 — Relationship View (20 min)

What you're building:

- For the selected datalist: show its parent and child datalists
- Show dynamic link targets (fields that point to other datalists)
- Data comes from the `all_relationships` MCP tool

Starting prompt (new conversation):

```
@project-rules.mdc
```

```
Checkpoint 3: Relationship view
```

```
For the selected datalist, show its relationships using the mCase MCP  
`all_relationships` tool.
```

```
The tool returns objects shaped as:
```

```
{ TargetDatalist, TargetField, SourceDatalist, SourceField, RelationshipType }  
RelationshipType is "ParentChild" or "Link"
```

```
Show:
```

1. Parent datalists: where RelationshipType="ParentChild" and TargetDatalist=selected datalist's SystemName
2. Child datalists: where RelationshipType="ParentChild" and SourceDatalist=selected datalist's SystemName
3. Linked datalists: where RelationshipType="Link" and either side matches

```
Display in a clear section below the field list.
```

```
Show relationship type and the related datalist name.
```

```
Handle the case where a datalist has no relationships (empty state).
```

```
Pause for review after the relationship section renders.
```

Review checklist before moving to CP4:

- ☐ Relationships render for datalists that have them
 - ☐ Empty state is handled gracefully (no crash, no blank space with no explanation)
 - ☐ Parent/Child vs. Link is visually distinct
 - ☐ Related datalist names are readable
-

Checkpoint 4 — Search (stretch, 10 min)

Only attempt if CP1-3 are solid and you have 10+ minutes remaining.

What you're building:

- A search input that filters the datalist list as you type
- Matches on Label and SystemName (case-insensitive)

Starting prompt (new conversation):

@project-rules.mdc

Checkpoint 4: Search

Add a search input that filters the datalist list as the user types.

- Filter on Label and SystemName (case-insensitive substring match)
- Client-side only – no additional MCP calls
- Clear button to reset the filter

Keep the implementation simple. No fuzzy matching needed.

Stretch Goal: Extend the MCP

If you completed all checkpoints and want to go deeper: add a new tool to the mCase MCP server.

Suggested tool: `datalist_stats` — returns field counts grouped by addType for a given datalist.

```
// Example return shape
{
  datalistId: number,
  datalistName: string,
  totalFields: number,
  requiredFields: number,
  fieldsByType: { [addType: string]: number }
}
```

The existing tools in `src/index.ts` are the pattern to follow.